

**VENTSPILS AUGSTSKOLA
INFORMĀCIJAS TEHNOLOĢIJU FAKULTĀTE**

BAKALaura DARBS

**VIRTUĀLĀS LABORATORIJAS IZVEIDE AR TĪMEKĻA
TEHNOLOĢIJĀM**

Autors

Ventspils Augstkolas

Informācijas tehnoloģiju fakultātes

bakalaura studiju programmas “Datorzinātnes”

3. kursa students

Ēvalds Zemturis

Matr. nr. 23020012

Fakultātes dekāns

doc. Dr.sc.comp. Vairis Caune

(paraksts)

Zinātniskais vadītājs

mg. oec. Gints Neimanis

(paraksts)

Recenzents

(paraksts)

Ventspils

2026

ANOTĀCIJA

Darba nosaukums: Virtuālās laboratorijas izveide ar tīmekļa tehnoloģijām

Darba autors: Ēvalds Zemturis

Darba vadītājs: mg. oec. Gints Neimanis

Darba apjoms:

Atslēgas vārdi: VIRTUALIZĀCIJA, TĪMEKĻA TEHNOLOĢIJAS,
LABORATORIJAS VIDES

Bakalaura darbā ir apskatītas vairākas eksistējošas populāras tehnoloģijas, kas ļauj īstenot virtuālu ierīču izveidi un piekļūt tām izmantojot pārlūkprogrammu vai citādākā veidā. Tiek apskatītas izmantotās tehnoloģijas, kā arī alternatīvas, kuras ir iespējams pielietot izmantoto tehnoloģiju vietā.

Darbā ir izstrādāta vide, kas apkopo vairākas tehnoloģijas, lai varētu parocīgā veidā izveidot, pārvaldīt un piekļūt dažādām virtuālu laboratoriju vidēm.

ANNOTATION

Title: Development of a virtual laboratory using web technologies

Author: Ēvalds Zemturis

Supervisor: mg. oec. Gints Neimanis

Volume:

Keywords: VIRTUALIZATION, WEB TECHNOLOGIES, LABORATORY ENVIRONMENTS

The Bachelor's thesis examines several existing popular technologies that enable the creation of virtual devices and provide access to them via a web browser or other means. The author analyzes the specific technologies utilized, as well as alternatives that could be implemented in their place.

As part of the work, an environment was developed that integrates multiple technologies to allow for the convenient creation, management, and accessing of various virtual laboratory environments.

SATURS

Virtuālās laboratorijas izveide ar tīmekļa tehnoloģijām.....	1
Anotācija.....	2
Annotation.....	3
Saīsinājumu un nosacīto apzīmējumu saraksts.....	5
Ievads.....	6
Tēmas aktualitāte.....	6
Darbā apskatāmās problēmas esošo risinājumu pārskats.....	6
Darba mērķis.....	6
Veicamie uzdevumi.....	6
Pētījuma realizācijas metodes un tehnoloģija.....	7
Darba struktūras apraksts.....	7
1. Tehnoloģiju teorētiskais apskats un alternatīvas.....	8
1.1 Virtualizācijas tehnoloģiju analīze (<i>Proxmox</i> un alternatīvas).....	8
1.2 Attālinātās piekļuves risinājumi (<i>Apache Guacamole</i> un alternatīvas).....	9
1.3. Sistēmas aizmugursistēmas izstrādes rīki (<i>Spring</i> satvars un alternatīvas).....	9
1.3.1. <i>Spring</i> satvara izvēles pamatojums.....	10
1.3.2. <i>Spring Boot</i> loma projektā.....	10
1.3.3. Alternatīvu izvērtējums.....	10
1.3.4. Secinājumi:.....	11
1.4. Datubāzu pārvaldības sistēmu analīze (<i>PostgreSQL</i> un alternatīvas).....	11
1.4.1. <i>PostgreSQL</i> raksturojums.....	11
1.4.2. Alternatīvu izvērtējums.....	11
1.4.3. Secinājums par izvēli.....	11
1.5. Infrastruktūras automatizācijas rīki (<i>Ansible</i> un alternatīvas).....	12
1.5.1. <i>Ansible</i> darbības principi un priekšrocības.....	12
1.5.2. Automatizācijas loma virtuālajā laboratorijā.....	12
1.5.3. Infrastruktūras automatizācijas alternatīvu izvērtējums.....	12
1.6. Tehnoloģisko alternatīvu kopsavilkums.....	13
2. Virtuālās laboratorijas sistēmas izstrāde.....	14
2.1. Sistēmas arhitektūras modelis.....	14
2.2. Datubāzes modelis un entītiņu projektēšana.....	14
2.3. Portālu izstrāde un funkcionalitāte.....	15
2.4. Infrastruktūras automatizācijas loģika.....	15
2.5. Autentifikācija un drošība.....	16
2.6. Sistēmas testēšana un kvalitātes novērtējums.....	16
2.6.1. Funkcionālā pārbaude un datu validācija.....	16
2.6.2. Infrastruktūras un piekļuves validācija.....	17
3. Rezultātu analīze.....	17
Izmantotās literatūras un avotu saraksts.....	18

SAĪSINĀJUMU UN NOSACĪTO APZĪMĒJUMU SARAKSTS

Saīsinājums/Nosacītais apzīmējums	Nozīme	Tulkojums latviešu valodā (ja nepieciešams)
API	Application Programming Interface	Lietojumprogrammas saskarne
SSH	Secure Shell (connection)	Drošas čaulas (savienojums)
RDP	Remote Desktop Protocol	Attālās darbvirsmas protokols
VNC	Virtual Network Computing	Virtuālā tīkla datošana
KVM	Kernel-based Virtual Machine	Uz kodola balstīta virtuālā mašīna
LXC	Linux Containers	<i>Linux</i> konteinerizācija
REST	Representational State Transfer	Reprezentatīva stāvokļa nodošana
CRUD	Create, Read, Update, Delete	Izveidot, nolasīt, atjaunināt, dzēst

IEVADS

Tēmas aktualitāte

Mūsdienu izglītības procesā, it īpaši tehnoloģiju jomā, pieaug nepieciešamība pēc elastīgas un viegli pieejamas mācību vides[1]. Darba tēmas izvēli pamato straujā virtualizācijas un attālināto savienojumu tehnoloģiju attīstība, kas ļauj efektīvi sadalīt jaudīgu sistēmu resursus mazākās, pilnvērtīgās virtuālās vienībās, kā arī pieslēgšanos tām. Šāda pieeja dod iespēju veikt resurintensīvas darbības, piemēram, koda kompilēšanu vai apjomīgu datu apstrādi, izmantojot attālās sistēmas jaudu, nevis klienta ierīces ierobežotos resursus. Automatizējot šos procesus un nodrošinot piekļuvi caur tīmekļa pārlūkprogrammu, augstākās izglītības iestādes var būtiski uzlabot mācību resursu pieejamību, padarot tos neatkarīgus no izglītojamā aparatūras nodrošinājuma vai tehniskajām priekšzināšanām[2].

Darbā apskatāmās problēmas esošo risinājumu pārskats

Šobrīd tirgū pastāv atsevišķi risinājumi gan virtualizācijai, gan attālinātai piekļuvei, gan procesu automatizācijai. Taču tie bieži vien nav optimizēti tieši izglītības iestāžu noteiktajām vajadzībām, radot vairākas praktiskas problēmas:

- **Laikietilpīga pārvaldība:** Daudzi profesionālie rīki pieprasa padziļinātas sistēmu administrēšanas zināšanas, kas pasniedzējiem rada lieku administratīvo slogu, novēršot uzmanību no mācību satura sagatavošanas.
- **Integrācijas trūkums:** Esošie risinājumi bieži vien ir sadrumstaloti – virtualizācija, lietotāju pārvaldība un attālinātā piekļuve darbojas kā atsevišķas sistēmas. Tas liedz izveidot vienotu vidi, kurā pasniedzējs varētu operatīvi sagatavot laboratorijas darbus un studenti tiem uzreiz piekļūt.
- **Atkarība no klienta aparatūras:** Bez centralizēta tīmekļa portāla studenti bieži vien saskaras ar tehniskām grūtībām, instalējot specifisku programmatūru uz dažādām operētājsistēmām, kas kavē mācību procesu un palielina tehnisko kļūdu risku.

Darba mērķis

Darba mērķis ir izveidot integrētu sistēmu, kas apvienos virtualizācijas, attālinātās piekļuves un automatizācijas tehnoloģijas, nodrošinot parocīgu platformu konfigurētu virtuālo laboratorijas vižu izveidei un pārvaldībai.

Veicamie uzdevumi

1. Veikt pieejamo tīmekļa un izstrādes tehnoloģiju izpēti virtuālo laboratoriju izveides kontekstā.
2. Izveidot virtuālo laboratoriju ģenerēšanas sistēmu ātrai un parocīgai dažāda mēroga darba vižu izveidei.
3. Izveidot pasniedzēja portālu tīmekļa vidē virtuālo ierīču un laboratorijas resursu pārvaldībai.
4. Izveidot studenta portālu piekļuvei piesaistītajiem uzdevumiem un attālinātai laboratorijas videi caur tīmekļa pārlūkprogrammu.
5. Realizēt laboratorijas vižu izveides un pārraudzības procesu automatizāciju, izmantojot mūsdienīgas konfigurācijas vadības tehnoloģijas.

Pētījuma realizācijas metodes un tehnoloģija

Pētījums balstīts uz lietišķo izstrādi, izmantojot sistēmu analīzes un projektēšanas metodes. Praktiskā risinājuma izstrādei izvēlētas šādas tehnoloģijas:

- Virtualizācijas platforma: Sistēmas pamatā tiek izmantota *Proxmox* virtuālā vide, kas nodrošina jaudīgu resursu pārvaldību un virtuālo mašīnu izolāciju
- Sistēmas aizmugursistēma: Portālu loģikas un API saskarņu izstrādei tiek izmantots *Spring* satvars, kas nodrošina mērogojamu un drošu lietojumprogrammas darbību.
- Automatizācija: Laboratorijas vižu automātiska izveide, konfigurēšana un pārvaldība tiek realizēta ar *Ansible* konfigurācijas vadības rīku, nodrošinot ātru un atkārtojamu infrastruktūras uzstādīšanu.
- Attālinātā piekļuve: Lietotāju piekļuve virtuālajām darba vidēm tīmekļa pārlūkprogrammā tiek nodrošināta ar *Apache Guacamole* vārteju. Tas ļauj piekļūt SSH, RDP un VNC sesijām bez papildu programmatūras instalēšanas klienta ierīcē.
- Saziņa starp komponentēm: Integrācija starp portālu, virtualizācijas serveri un attālinātās piekļuves vārteju tiek nodrošināta, izmantojot API risinājumus un attālinātās darbvirsmas protokolus.

Darba struktūras apraksts

Bakalaura darbs sastāv no trim nodaļām

- Pirmajā nodaļā tiek apskatītas dažādas tīmekļa un izstrādes tehnoloģijas ar kurām tika pilnveidots bakalaura darbs, kā arī dažas to alternatīvas
- Otrajā nodaļā ir aprakstīta praktiskā daļas izstrāde, detalizēti dokumentējot sistēmas arhitektūras un portālu izveides procesu.
- Trešajā nodaļā tiek demonstrēts izstrādātais gala rezultāts, analizēta atbilstība izvirzītajām prasībām
- Darba noslēgumā apkopoti secinājumi un sniegti priekšlikumi sistēmas turpmākajai pilnveidei.

1. TEHNOLOĢIJU TEORĒTISKAIS APSKATS UN ALTERNATĪVAS

Pirmā nodaļa tiek veltīta virtuālo laboratoriju izveides nepieciešamo tehnoloģiju analīzei. Darba autors apskata populārākās eksistējošās tehnoloģijas, kas ļauj īstenot virtuālo ierīču pārvaldību un piekļuvi tām izmantojot tīmekļa pārlūkprogrammu.

1.1 Virtualizācijas tehnoloģiju analīze (*Proxmox* un alternatīvas)

Virtualizācija nodrošina fizisko resursu segmentāciju, radot izolētas mācību vides. Galvenais izaicinājums šajā etapā ir izvēlēties hipervizoru, kas piedāvā stabilu API saskarni automatizācijai.

Proxmox VE: Autora izvēlētā platforma ir atvērtā pirmkoda risinājums, kas balstīta uz *Debian Linux*. [3]

Priekšrocības: Apvieno divu veidu virtualizāciju – KVM pilnai virtualizācijai un LXC konteinerizācijai. KVM nodrošina pilnu aparatūras virtualizāciju, kas nepieciešama dažādu operētājsistēmu (piemēram, *Windows*) darbināšanai, savukārt LXC ir efektīvāka resursu ziņā un piemērota identisku *Linux* vižu ātrai pavairošanai. Nodrošina iebūvētu tīmekļa saskarni un jaudīgu API, kas ir būtiski sistēmas integrācijai ar ārējiem portāliem. [3]

Trūkumi: Tīkla konfigurācija sarežģītākos scenārijos var prasīt padziļinātas *Linux* zināšanas.

Alternatīvas:

XCP-ng un Xen Orchestra: *Xen Cloud Platform – next generation* hipervizoru kopā ar *Xen Orchestra* pārvaldes vidi ir labs atvērtā pirmkoda risinājums, kas ir balstīts uz

Xen Project[4], nevis uz *Debian* bāzes kā *Proxmox VE*. Tas izceļas ar augstu drošības līmeni un stabilu veiktspēju liela mēroga infrastruktūrās.

Salīdzinājums: Lai gan *XCP-ng* ir tehniski nobriedis risinājums, bakalaura darbā izvēle par labu *Proxmox VE* izdarīta darba autora iepriekšējās pieredzes dēļ.

1.2. Attālinātās piekļuves risinājumi (*Apache Guacamole* un alternatīvas)

Lai nodrošinātu lietotāju piekļuvi virtuālajām darba vidēm tiešā veidā caur tīmekļa pārlūkprogrammu, neizmantojot specifisku programmatūru (piemēram, *Putty* vai RDP klientus) ir nepieciešama starpniekservera jeb vārtejas tehnoloģija.

Apache Guacamole: Autora izvēlētā tehnoloģija ir bezklienta attālinātās darbvirsmas vārteja. Tā atbalsta standarta attālinātās pieslēgšanas protokolus kā VNC, RDP un SSH.

Darbības princips: *Guacamole* serveris (*guacd*) sazinās ar virtuālo ierīc, izmantojot tās vietējo protokolu, un pārveido grafisko izvadi uz HTML5 *Canvas* elementu, kas ir saprotams jebkurai mūsdienu pārlūkprogrammai.[5]

Priekšrocības: Galvenais ieguvums ir “bez instalācijas” princips klienta pusei, kas ļauj studentiem piekļūt laboratorijas resursiem no jebkuras ierīces, neatkarīgi no operētājsistēmas izvēles[5]. Papildus ieguvums ir izolācija no virtuālās ierīces, jo lietotājs nekad neizveido tiešu savienojumu – savienojas tikai ar vārteju.

Integrācija: *Guacamole* projekts piedāvā atvērtu API, kas ļauj to integrēt ar citām sistēmām, šajā gadījumā – ar *Spring* satvara izveidoto studentu un pasniedzēju portālu *Java* programmēšanas valodā[5].

Alternatīvas: Darba ietvaros tika izskatītas alternatīvas, kas nodrošina līdzīgu funkcionalitāti. Viena no tām ir *noVNC* klients, kas ir integrēts pašā *Proxmox* saskarnē. Taču tas nepiedāvā tik plašas sesiju pārvaldības un protokolu apvienošanas iespējas vienotā vārtejā kā *Apache Guacamole*[5][6].

Secinājums: Izvēle par labu *Apache Guacamole* ir pamatota ar tās spēju apvienot dažādus protokolus vienā saskarnē un integrāciju ar sistēmas aizmugursistēmu, izmantojot API risinājumus.

1.3. Sistēmas aizmugursistēmas izstrādes rīki (*Spring* satvars un alternatīvas)

Sistēmas aizmugursistēmas izstrādei tika izvēlēts *Spring* satvars, kas ir viena no populārākajām atvērtā pirmkoda platformām uz Java programmēšanas valodas bāzes[7]. Izvēle par labu šai tehnoloģijai balstīta uz vairākiem faktoriem, kas ietekmē sistēmas stabilitāti un izstrādes ātrumu.

1.3.1. *Spring* satvara izvēles pamatojums

- **Personīgā pieredze:** Viens no galvenajiem argumentiem par labu *Spring* izvēlei ir autora iepriekšējā praktiskā pieredze ar šo satvaru. Tas ļauj efektīvāk izmantot sistēmas arhitektūras iespējas, nodrošināt kvalitatīvāku kodu, kā arī samazināt izstrādes procesā pieļauto kļūdu skaitu.
- **Vienkārša integrācija ar *Apache Guacamole*:** Izstrādājot virtuālo laboratoriju, kritiski svarīga ir aizmugursistēmas spēja sazināties ar attālinātās piekļuves vārteju. *Spring* satvars piedāvā lieliskas bibliotēkas un rīkus *RESTful* API izstrādei, kas būtiski atvieglo integrāciju ar *Apache Guacamole* atvērto API. Tas ļauj programmātiski pārvaldīt lietotāju sesijas, autentifikācijas žetonus un piekļuves tiesības vienotā ekosistēmā.
- **Mērogojamība un drošība:** *Spring* satvars nodrošina mērogojamu un drošu lietojumprogrammas darbību, kas ir būtiski, apkalpojot vairākus lietotājus vienlaicīgi un pārvaldot piekļuvi sensitīviem infrastruktūras resursiem.

1.3.2. *Spring Boot* loma projektā

Darbā tiek izmantots *Spring Boot*, kas vienkāršo lietojumprogrammu konfigurēšanu un izvietošānu. Tā galvenā funkcija ir kalpot kā starpslānim starp lietotāja saskarni un zemāka līmeņa tehnoloģijām (*Proxmox*, *Guacamole*).

- **API saskarņu izstrāde:** Ar *Spring Boot* palīdzību tiek izveidotas API saskarnes, kas nepieciešamas studentu un pasniedzēju portālu loģikas realizēšanai.

1.3.3. Alternatīvu izvērtējums

Lai gan *Spring* satvars ir izvēlēts autora personīgās pieredzes dēļ, tika izskatītas arī citas tehnoloģijas:

- **Node.js (Express):** Piedāvā ātru prototipēšanu un asinhronu datu apstrādi. Tomēr liela mēroga un stingri tipizētu sistēmu izstrādē *Java* un *Spring* satvars piedāvā stabilāku struktūru, kas samazina loģikas kļūdu risku.[8]

- **Python (Django/FastAPI):** Šie satvari piedāvā izcilu integrāciju ar automatizācijas rīkiem (piemēram, *Ansible*), jo tie balstīti tajā pašā programmēšanas valodā – *Python*. Tomēr salīdzinot ar autora izvēlēto *Spring* satvaru, tie ne vienmēr nodrošina tikpat stingru arhitektūras struktūru liela mēroga uzņēmuma līmeņa aplikācijām

1.3.4. Secinājumi:

Spring satvara izvēle, apvienojumā ar autora pieredzi un tehnoloģijas dabisko saderību ar *Apache Guacamole*, nodrošina stabilu un viegli uzturamu pamatu visas virtuālās sistēmas darbībai.

1.4. Datubāzu pārvaldības sistēmu analīze (*PostgreSQL* un alternatīvas)

Lai nodrošinātu lietotāju datu, kursu struktūru un laboratorijas sesiju informācijas pastāvīgu glabāšanu, nepieciešama uzticama un mērogojama datubāzu pārvaldības sistēma. Autors darba ietvaros ir izvēlējies *PostgreSQL* datubāzu pārvaldības sistēmu.

1.4.1. *PostgreSQL* raksturojums

PostgreSQL ir atvērta pirmkoda objektu relāciju datubāzu sistēma.

- **Priekšrocības:** Tā izceļas ar augstu atbilstību SQL standartiem, jaudīgu transakciju vadību (ACID) un plašu atbalstu kompleksiem datu tipiem. Izstrādes procesā *PostgreSQL* nodrošina stabilu savienojamību ar *Spring Data JPA* repozitorijiem, kas atvieglo entītiju kartēšanu un CRUD operāciju veikšanu.
- **Papildus ieguvumi:** Tā kā sistēma ir atvērta pirmkoda, tā nerada papildu izmaksas infrastruktūras uzturēšanai, kas ir svarīgi izglītības iestādēm.

1.4.2. Alternatīvu izvērtējums

Izvēloties piemērotāko risinājumu, tika izskatītas vairākas alternatīvas:

- **MariaDB[9]:** Populāra atvērta pirmkoda relāciju datubāze, kas izveidota kā *MySQL* atzarojums. Lai gan *MariaDB* piedāvā augstu veiktspēju un savietojamību, *PostgreSQL* tika dota priekšroka tās jaudīgāko funkciju dēļ, kas saistītas ar sarežģītu vaicājumu optimizāciju un paplašinātu datu integritātes nodrošināšanu.
- **MongoDB (NoSQL)[10]:** Dokumentu orientēta datubāze, kas nodrošina lielu elastību datu shēmas maiņā. Tomēr, ņemot vērā, ka šajā projektā datiem (piemēram, studentu sasaistei ar kursiem un pasniedzējiem) ir izteikta relāciju struktūra, piemērotāks risinājums ir relāciju datubāzes sistēma nekā *NoSQL* risinājums.

1.4.3. Secinājums par izvēli

Izvēle par labu *PostgreSQL* ir pamatota ar tās uzticamību, izstrādes gaitā gūto pozitīvo pieredzi savienojuma izveidē ar *Spring Boot* un spēju efektīvi atbalstīt lomu bāzētu piekļuves vadības sistēmu, kas ir viena no darba pamatprasībām.

1.5. Infrastruktūras automatizācijas rīki (*Ansible* un alternatīvas)

Lai nodrošinātu ātru un atkārtojamu laboratorijas vižu izveidi, darbā tiek izmantota “infrastruktūra kā kods” pieeja. Manuāla virtuālo mašīnu konfigurēšana ir laikietilpīgs process un pakļaujas uz cilvēciskām kļūdām, tādēļ automatizācijai izvēlēts *Ansible* rīks.

1.5.1. Ansible darbības principi un priekšrocības

Ansible ir atvērtā pirmkoda konfigurācijas vadības rīks, kas izceļas ar savu “bez aģenta” arhitektūru.[11]

- **Saziņas protokols:** Atšķirībā no citiem rīkiem, *Ansible* nav nepieciešams instalēt papildu programmatūru klienta ierīcēs. Saziņa notiek caur SSH protokolu.
- **Deklaratīvā valoda:** Konfigurācijas scenāriji jeb “playbooks” tiek sarakstīti cilvēkiem salasāmā *YAML* formātā, kas atvieglo to uzturēšanu un pārskatāmību.
- **Idempotence:** Šī īpašība nodrošina, ka, izpildot vienu un to pašu scenāriju atkārtoti, sistēma netiek bojāta un tiek sasniegts tikai vēlamais gala stāvoklis.

1.5.2. Automatizācijas loma virtuālajā laboratorijā

Virtuālās laboratorijas ietvaros *Ansible* veic šādus uzdevumus:

1. **Sākotnējā konfigurācija:** Virtuālo ierīču izveide, tīkla iestatījumu pielāgošana
2. **Programmatūras uzstādīšana:** Nepieciešamo rīku (piemēram, kompilatoru, datubāzu vai teksta redaktoru) instalēšana atkarībā no laboratorijas darba tēmas.

1.5.3. Infrastruktūras automatizācijas alternatīvu izvērtējums

Izvēloties rīku laboratorijas vižu automatizētai izveidei un konfigurēšanai, tika izskatītas vairākas nozarē populāras alternatīvas:

- **Terraform[12]:**
 - **Apraksts:** Rīks, kas fokusējas uz infrastruktūras izveidi, izmantojot deklaratīvu pieeju
 - **Salīdzinājums:** Lai gan *Terraform* ir izcils rīks virtuālo ierīču izveidei Proxmox vidē, tam trūkst elastības operētājsistēmas iekšējā konfigurēšanā

(lietotņu instalēšana, failu rediģēšana), kas ir būtiski laboratorijas darba sagatavošanā.

- **Bash skripti:**

- **Apraksts:** Vienkārša un tieša automatizācijas metode, izmantojot čaulas skriptus.
- **Salīdzinājums:** Skriptiem trūkst idempotences īpašība – spējas droši palaist vienu un to pašu kodu vairākas reizes, neizraisot kļūdas vai dublētus ierakstus. *Ansible* nodrošina drošā un strukturētāku pieeju, kas ir svarīga atkārtojamu laboratorijas vižu izveidei.

Secinājums par automatizācijas izvēli:

Ansible tika izvēlēts, jo tas spēj nodrošināt pilnu automatizācijas ciklu – gan virtuālo mašīnu izveidi caur *Proxmox* API, gan to tālāku konfigurēšanu caur SSH. Tas ir optimālākais risinājums, lai izveidotu ātru un parocīgu dažāda mēroga darba vižu ģenerēšanas sistēmu.

1.6. Tehnoloģisko alternatīvu kopsavilkums

Izvērtējot 1. nodaļā analizētās tehnoloģijas, tika meklēts optimāls līdzsvars starp sistēmas jaudu, lietojamības ērtumu un izstrādes laiku. Zemāk esošajā tabulā apkopotas izvēlētās tehnoloģijas un to galvenās alternatīvas.

1.1. tabula

Tehnoloģisko alternatīvu kopsavilkuma tabula

Sastāvdaļa	Izvēlētā tehnoloģija	Alternatīvas	Izvēles pamatojums
Datu glabāšana	PostgreSQL	MariaDB, MongoDB	Augsta datu integritāte, ACID atbilstība un autora personīgā interese
Virtualizācija	Proxmox VE	XCP-ng	Iebūvēts KVM un LXC atbalsts, spēcīgs API un autora pieredze
Attālinātā piekļuve	Apache Guacamole	noVNC	Atalsta vairākus protokolus (SSH, RDP, VNC) bez papildu spraudņiem
Aizmugursistēma	Spring Boot	Node.js, Django	Augsta drošība (Spring Security),

			ērta API integrācija, autora pieredze
Automatizācija	Ansible	Terraform	“Bez aģenta” arhitektūra un vienkārša konfigurācijas vadība caur SSH

Secinājums: Izvēlētais tehnoloģiju kopums nodrošina pilnu dzīves ciklu virtuālajai laboratorijai – no resursu izolācijas līdz automatizētai piegādei un ērtai piekļuvei pārlūkprogrammā.

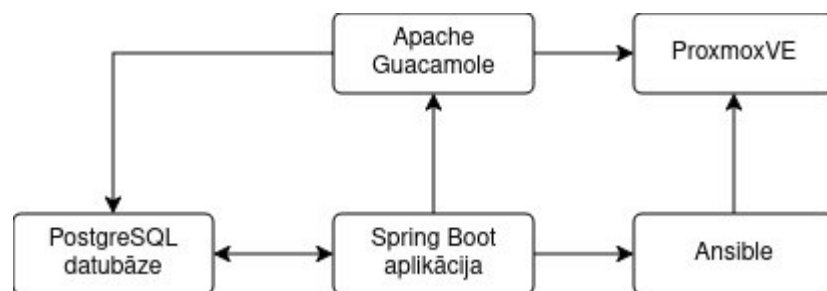
2. VIRTUĀLĀS LABORATORIJAS SISTĒMAS IZSTRĀDE

Šajā nodaļā tiek aprakstīts praktiskais izstrādes process, kas balstīts uz iepriekš analizētajām tehnoloģijām, lai sasniegtu darba mērķi – izveidotu integrētu platformu Linux sistēmu apguvei.

2.1. Sistēmas arhitektūras modelis

Sistēma ir veidota, izmantojot daudzslāņu arhitektūru, kur centrālo lomu ieņem *Spring Boot* lietojumprogramma. Tā darbojas kā starpslānis, kas nodrošina saziņu starp:

- **Lietotāja saskarni** (studentu un pasniedzēju portāliem)
- **Virtualizācijas serveri (*Proxmox VE*)**, izmantojot tā API resursu pārvaldībai
- **Attālinātās piekļuves vārteju (*Apache Guacamole*)** sesiju nodrošināšanai
- **PostgreSQL datubāzi** datu pastāvīgai glabāšanai

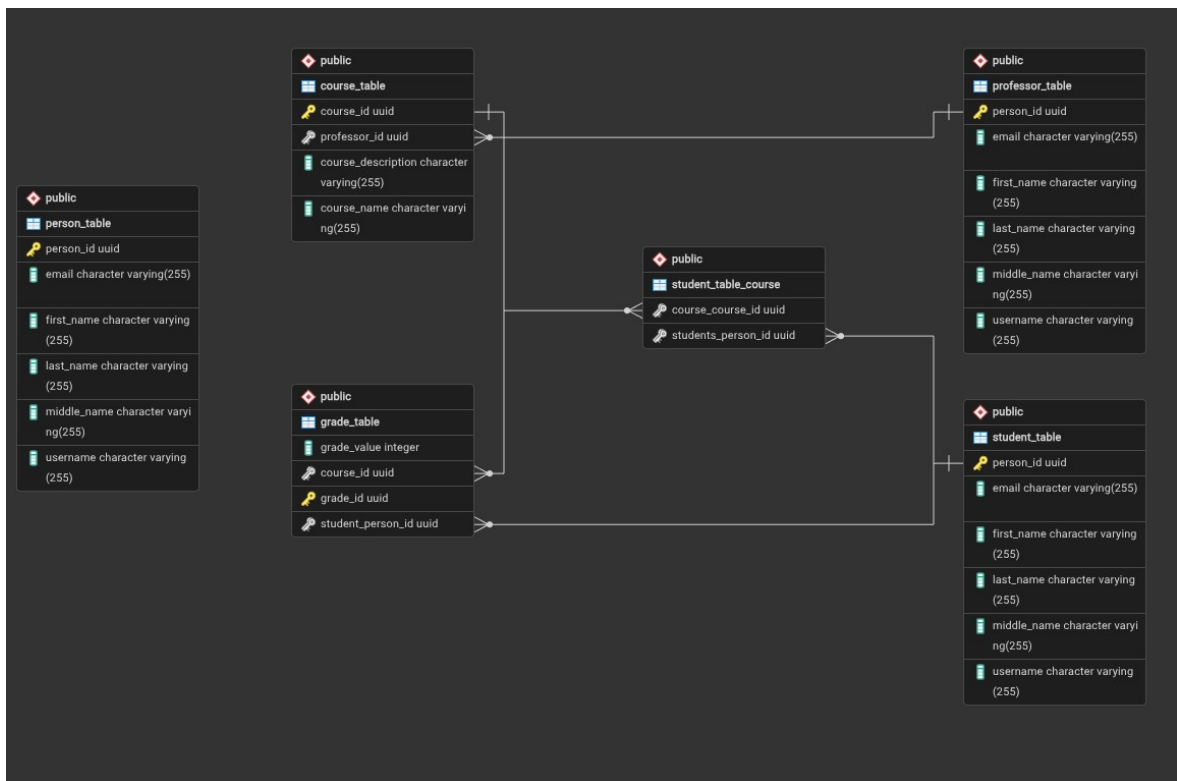


2.1. att. Sistēmas arhitektūras diagramma

2.2. Datubāzes modelis un entītiju projektēšana

Sistēmas datu struktūra ir izstrādāta, lai atbalstītu lomu balstītu piekļuves vadību un mācību procesa pārvaldību. Galvenās datu bāzes entītijas, kas tika izveidotas izstrādes procesā ir:

- **Person (Persona):** Pamata klase lietotāju informācijai
- **Student (Students)** un **Professor (Pasniedzējs):** Specializētāki lietotāju modeļi ar attiecīgajām tiesībām
- **Course (Kurss):** Modelis, kas apvieno studentus un mācību materiālus
- Datu piekļuvei tiek izmantoti *Spring Data JPA* repozitoriji, kas ļauj veikt CRUD operācijas ar datubāzes objektiem.



2.2. att. Datubāzes entītiju saišu diagramma

2.3. Portālu izstrāde un funkcionalitāte

Praktiskajā daļā tika realizēti divi galvenie piekļuves punkti:

1. **Pasniedzēju portāls:** Nodrošina sākumlapu, kontrolierus un servissus virtuālo ierīču un laboratorijas resursu pārvaldībai. Realizētas CRUD operācijas profesoru un kursu datiem.

2. **Studentu portāls:** Nodrošina piekļuvi piesaistītajiem uzdevumiem un attālinātajai videi caur pārlūku, bez papildus programmatūras instalācijas. Studentu datu pārvaldībai arī izveidots atbilstošs CRUD prototips.

2.4. Infrastruktūras automatizācijas loģika

Lai nodrošinātu ātru un atkārtojamu laboratorijas vižu izveidi, tika izstrādāti *Ansible* scenāriji jeb “playbook”. Automatizācijas process ietver:

- Virtuālo mašīnu automātisku konfigurēšanu *Proxmox* vidē.
- Nepieciešamo rīku (kompilatoru, servisu) instalēšanu atkarībā no uzdevuma specifikas.
- Tīkla iestatījumu pielāgošanu drošai attālinātai piekļuvei.

[ANSIBLE KODA FRAGMENTS]

2.5. Autentifikācija un drošība

Sistēmā ir integrēta autorizēšanās sistēma, kas pēc pieteikšanās piešķir lietotājam noteiktas tiesības (piemēram, studentam – piekļuve savai darba videi, pasniedzējam – visu darbu novērtēšana). Izstrādei izmantots *Spring Security* satvars, nodrošinot drošu sesiju vadību un integrāciju ar *Apache Guacamole* API.

2.6. Sistēmas testēšana un kvalitātes novērtējums

Lai pārliecinātos par virtuālās laboratorijas sistēmas stabilitāti un atbilstību izvirzītajiem uzdevumiem, tika veikts vairāku līmeņu testēšanas process, aptverot gan programmatūras kodu, gan infrastruktūras automatizācijas loģiku.

2.6.1. Funkcionālā pārbaude un datu validācija

Tā kā sistēmas izstrādes posmā galvenais uzsvars tika likts uz prototipa funkcionalitāti, testēšana tika veikta manuāli, pārbaudot katra moduļa darbību pēc tā izveides:

- **Datubāzes integritātes pārbaude:** Pēc savienojuma izveides ar *PostgreSQL* un modeļu klašu definēšanas tika veikta tieša datu ievades un nolasīšanas pārbaude datubāzē. Tas ļāva pārliecināties, ka *Spring Data JPA* repozitorijs pareizi kartē *Java* objektus uz SQL tabulām.
- **CRUD operāciju manuālā testēšana:** Izmantojot izstrādāto studentu un profesoru portālu saskarni, tika veiktas pārbaudes, lai apstiprinātu lietotāju datu

pievienošanu, apskati, labošanu un dzēšanu. Šis process nodrošināja, ka kontrolieri un servisi pareizi apstrādā lietotāja ievadīto informāciju.

- **Testa datu izmantošana:** Lai pārbaudītu sistēmas darbību dažādos scenārijos, tika izveidots specifisks testa datu kopums (piemēram, vairāki kursi un tiem piesaistīti studenti), kas ļāva pārbaudīt sasaistes loģiku starp entitijām.

2.6.2. Infrastruktūras un piekļuves validācija

Papildus lietojumprogrammas kodam tika veikta infrastruktūras automatizācijas un attālinātās piekļuves pārbaude:

- **Ansible scenāriju validācija:** Izveidotie *Ansible* scenāriji tika pārbaudīti uz testa virtuālajām mašīnām, verificējot to idempotenci – spēju sasniegt vēlamu konfigurācijas stāvokli, neradot kļūdas pie atkārtotas palaišanas.

3. REZULTĀTU ANALĪZE

3.1. Izstrādāto portālu darbības demonstrācija

Praktiskās izstrādes rezultātā ir izveidotas divas galvenās lietotāju saskarnes:

- **Pasniedzēju portāls:** Tas nodrošina centralizētu vidi kursu un virtuālo resursu pārvaldībai. Portālā realizētās CRUD operācijas ļauj pasniedzējam pievienot jaunus studentus, veidot mācību kursus un caur integrēto API ietekmēt *Proxmox* vidē esošās virtuālās mašīnas.
- **Studentu portāls:** Nodrošina studentiem personalizētu skatu uz piesastūtajiem kursiem un uzdevumiem. Galvenais rezultāts ir sekmīga piekļuve virtuālajai darba videi tieši pārlūkprogrammā

3.2. Funkcionalitātes atbilstība darba uzdevumiem

- Virtualizācijas integrācija
- Attālinātā piekļuve
- Infrastruktūras automatizācija
- Drošība un lomas

3.3 Sistēmas ieguvumu un ierobežojumu novērtējums

....

4. SECINĀJUMI UN PRIEKŠLIKUMI

Secinājumi

- Izstrādātais sistēmas prototips apliecina, ka izvēlētais tehnoloģiju steks (*Spring Boot*, *Proxmox*, *Guacamole*, *Ansible*) ir pilnībā savietojams integrētas virtuālās laboratorijas izveidei.
- Izmantojot *Ansible* “infrastruktūra kā kods” pieeju, ļauj daudzkreiz samazināt veltīto laiku, kas nepieciešams jaunu laboratorijas darbu sagatavošanai.

Priekšlikumi

- Ieviest automatizēto vienību un integrācijas testēšanu aizmugursistēmas koda kvalitātes tālākai uzlabošanai
- Paplašināt *Ansible* scenāriju bibliotēku
- Integrēt sistēmas monitoringa rīkus, lai pasniedzējs varētu sekot studentu virtuālo ierīču resursu patēriņam.

IZMANTOTĀS LITERATŪRAS UN AVOTU SARAKSTS

- [1] Sivakumar, Shyamala C and Robertson, William and Artimy, Maen and Aslam, Nauman, *A web-based remote interactive laboratory for Internetworking education*, 2005.
https://www.researchgate.net/profile/S-Sivakumar-9/publication/3051052_A_Web-Based_Remote_Interactive_Laboratory_for_Internetworking_Education/links/09e41513600f9a3395000000/A-Web-Based-Remote-Interactive-Laboratory-for-Internetworking-Education.pdf
- [2] Swargiary, Khritish, *Enhancing Educational Accessibility*, 2024.
https://www.preprints.org/frontend/manuscript/9401940c8e9aa81504ca429e58e362d3/download_pub
- [3] Proxmox VE contributors, *Proxmox VE Reference Documentation - Introduction*, 2025. <https://pve.proxmox.com/pve-docs/chapter-pve-intro.html>
- [4] Vates SAS, Linux Foundation, *XCP-ng Documentation*, 2026. <https://docs.xcp-ng.org/>
- [5] The Apache Software Foundation, *Apache Guacamole Manual v1.6.0*, 2025.
<https://guacamole.apache.org/doc/gug/guacamole-architecture.html#implementation-and-architecture>
- [6] noVNC contributors, *noVNC API documentation*, 2025.
<https://github.com/novnc/noVNC/blob/master/docs/API.md>
- [7] Olivieri, Carlos Eduardo, *The Most Used Java Frameworks Nowadays* (2026), 2026. <https://higher-order-programmer.medium.com/the-most-used-java-frameworks-nowadays-2026-4d11264a1540>
- [8] Choma, Dominik and Chwaleba, Kinga and Dzie{\n}kowski, Mariusz, *The efficiency and reliability of backend technologies: express, Django, and spring boot*, 2023. https://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-f9590518-b6aa-40fb-8f3d-f9e796852aaf/c/Choma_The_4_23.pdf
- [9] MariaDB Foundation, *About MariaDB Server*, 2026.
<https://www.mariadb.org/about/>
- [10] MongoDB, Inc., *Mongo DB documentation*, .
<https://www.mongodb.com/docs/>
- [11] Red Hat, *What is Ansible?*, 2025. <https://www.redhat.com/en/ansible-collaborative?intcmp=7015Y000003t7aWQAQ>
- [12] HashiCorp, *What is Terraform?*, .
<https://developer.hashicorp.com/terraform/intro>